ДИСЦИПЛИНА **Математическая логика и теория алгоритмов**
(полное наименование дисциплины без сокращений)

ИНСТИТУТ **информационных технологий**

КАФЕДРА **прикладной математики**
(полное наименование кафедры)

ВИД УЧЕБНОГО МАТЕРИАЛА **Лекция**
(в соответствии с пп.1-11)

ПРЕПОДАВАТЕЛЬ **Батенков Кирилл Александрович**
(фамилия, имя, отчество)

СЕМЕСТР **1, 2024-2025**
(указать семестр обучения, учебный год)

1. СЛОЖНОСТЬ АЛГОРИТМОВ, ЧАСТЬ 1

Чистая математика изучает какие угодно объекты и функции и ставит любые вопросы о них [32]. Важную роль играют бесконечные объекты: действительные и комплексные числа, топологические пространства, ординалы и т.п. Ответ на поставленный вопрос часто может быть неконструктивен: например, существование объекта доказывается без конкретной процедуры его поиска.

Классическая теория вычислимости накладывает определённые рамки. Во-первых, изучаются лишь конечные объекты, то есть такие, которые можно закодировать словом в некотором алфавите или натуральным числом. Это могут быть сами числа, конечные их наборы, матрицы, графы и другие подобные структуры. Во-вторых, из всех функций над такими объектами рассматриваются лишь вычислимые. Это естественное ограничение: ведь и у функции (бесконечного объекта) тоже должно быть конечное описание. В-третьих, решаемыми задачами считаются не те, где решение есть в принципе, а лишь те, где решение можно найти при помощи алгоритма. Для построение связной теории важно, что есть общепринятое понятие алгоритма, не зависящее от низкоуровневых деталей реализации. Сами реализации могут быть самыми разными: машины Тьюринга и их вариации, адресные машины, рекурсивные функции, лямбдакомбинаторы и т.д.

Теория сложности вычислений идёт ещё на шаг дальше: решаемыми задачами считаются лишь те, у которых есть эффективное решение, то есть разрешающий алгоритм, работающий не слишком долго.

В английском языке есть два разных слова, означающие эффективность; effectiveness и efficiency. Первое из них означает, что задача в принципе решается, т.е. речь идёт о разрешимом множестве или вычислимой функции. Второе означает эффективность в вышеуказанном смысле.

Основной (и нерешённый) вопрос состоит в том, как отличить решаемую задачу от нерешаемой. Для начала его нужно чётко поставить.

1.1. Вычислительная сложность алгоритмов

Вначале мы очертим крут задач, о которых идёт речь [32]. Во-первых, мы будем изучать только задачи про конечные объекты. Вообще говоря, можно изучать сложность вычислительных задач и для бесконечных объектов. Так, ещё до создания современных компьютеров строились аналоговые вычислительные машины. Если мышление человека рассматривать как вычислительный процесс, то он именно аналоговый. Многие численные методы, например, метод Ньютона, естественно рассматривать как алгоритмы над действительными числами. Есть и специальная область теории алгебраические вычисления. Тем не менее, все эти аспекты выходят за рамки нашего повествования.

Под конечными объектами мы будем понимать объекты, которые можно «эффективно» закодировать словами в некотором алфавите (без ограничения общности, в двоичном). Эффективность кодирования означает, что по коду объекта можно вычислить все ключевые характеристики. Например, по коду графа можно вычислить, сколько в нём вершин и какие из них соединены рёбрами, по коду матрицы можно вычислить её размер и число в любой ячейке, и т.д. Как правило, кодирование не представляет особых проблем: с обычными строками, описывающими объекты, можно работать алгоритмически. Например, натуральные числа представляются своей двоичной (или десятичной) записью, вектора и матрицы представляются как цепочки записей чисел с разделителями, граф можно представить в виде списка рёбер или в виде матрицы смежности, и т.д. Во всех примерах мы будем считать, что все объекты уже закодированы как слова в некотором алфавите, и не будем отдельно обозначать код объекта. Разумеется, исключением будут бесконечные объекты, такие как действительные числа или геометрические фигуры. Такие объекты мы рассматривать не будем, хотя содержательная теория возможна и для них.

Мы будем изучать массовые задачи, т.е. бесконечные семейства индивидуальных задач для всевозможных конкретных значений параметров. Алгоритм должен действовать по одной и той же инструкции для всех экземпляров. У каждого алгоритма должен быть вход и выход. В зависимости от того, как они выглядят, мы будем различать несколько основных видов вычислительных задач:

1. Задачи поиска. Наверное, это первая постановка, которая приходит на ум при слове «задача»: дано условие, нужно найти решение. Например, требуется решить уравнение, или найти кратчайший маршрут в графе, или составить расписание согласно требованиям, или придумать, как уместить все детали на один лист фанеры. Формально задано отношение $R \subset \Sigma^* \times \Sigma^*$ и слово x, а найти нужно слово y, такое что xRy, если такое слово есть. Иными словами, алгоритм на любом x должен возвратить либо y, для которого xRy, либо $\perp$, если таких y нет.

2. Задача распознавания языка. Может быть так, что итоговый ответ бинарный: да шли нет. Например, является ли число простым, подходит ли пароль к сохранённому хеш-значению, легитимна ли транзакция в платёжной системе, горит ли зелёный на светофоре. В общем случае по слову x нужно выяснить, лежит ли оно в языке $L \subset \Sigma^*$. Иными словами, алгоритм должен возвратить 1 на любом $x \in L$ и возвратить 0 на любом $x \notin L$. Можно также сказать, что алгоритм должен проверить, выполнено ли для слова x данное свойство.

3. Задача вычисления функции. Если в задаче поиска всегда ровно один ответ, то удобно воспринимать его как функцию от входа. Это может быть отсортированный массив, значение арифметического выражения, разложение на простые множители. В общем случае задана некоторая функция $f: \Sigma^* \to \Sigma^*$, а алгоритм должен на любом x возвратить $f(x)$. Как правило, мы будем считать, что f всюду определена.

4. Задача подсчёта. Бывает так, что важно посчитать количество решений: например, сколько может быть различных меню в столовой, всем требованиям и возможностям, чтобы каждый день было новое меню. Либо в результате тонкого анализа оказывается, что значение функции есть число решений некоторой задачи. В общем случае для фиксированного отношения $R \subset \Sigma^* \times \Sigma^*$ по слову x нужно найти количество слов y, таких что xRy.

5. Задача оптимизации. Это ещё один вид задач, часто встречающийся на практике. Люди и фирмы всё время что-то оптимизируют: минимизируют время выполнения задания или затраченные деньги, максимизируют прибыль или удовольствие. Вообще говоря, задача оптимизация есть частный случай задачи поиска: по слову x нужно найти такой y, что значение $f(x, y)$ максимально (или минимально), где $f: \Sigma^* \times \Sigma^* \to \mathbb{Q}$ - некоторая числовая функция. Альтернативно, может требоваться найти лишь само оптимальное значение.

6. Задача аппроксимации. На практике не требуется найти настоящий оптимум, приближённое решение тоже хорошо подойдёт. В теории задача ставится так: по слову x нужно найти такой y, что значение $f(x, y)$ составляет хотя бы $(1 - \varepsilon)$ долю от максимального (нли не более чем в $(1 + \varepsilon)$ раз больше минимального). Вновь может требоваться найти лишь само приближённо оптимальное значение.

Для алгоритмического решения задач требуются различные вычислительные ресурсы. Некоторые из них сразу приходят на ум, другие доступны только в теории:

1. Время работы. Как правило, задачу нужно решить не в принципе, а к определённому сроку. Заведомой теоретической верхней границей является предполагаемый срок дальнейшего существования Вселенной, но обычно дедлайн куда ближе. Поэтому время работы – важнейшая характеристика программы. Поскольку мы изучаем алгоритмы, выполняемые пошагово, то время удобно мерить в этих шагах.

2. Память. Алгоритму необходимо рабочее пространство для промежуточных вычислений. Обычно в это пространство не включают вход программы: например, при поиске информации среди всех файлов диска разумно подсчитывать только оперативную память, выделенную под этот процесс, и временные файлы, а не общий объём диска.

3. Случайность. Некоторые задачи (например, проверка числа на простоту или проверка двух арифметических выражений на равенство) при современном уровне знаний решаются гораздо быстрее, если применить вероятностный алгоритм вместо детерминированного. Как правило, расплатой за это ускорение будет маленькая, но положительная вероятность ошибки. Помимо самого факта использования случайных битов, важными характеристиками являются «количество» и «качество» использованной случайности. Под количеством понимается число случайных битов, под качеством - близость их распределения к равномерному, в том числе малые корреляции между значениями разных битов.

4. Подсказка. Если машина получает небольшую подсказку, зависящую только от длины входа, она может существенно ускориться. Более того, с подсказкой можно решить даже некоторые невычислимые функции. Использование подсказки моделирует ситуацию, в которой длина входа заранее известна и можно сконструировать микросхему ровно для этой длины. Например, функционирование криптовалюты Bitcoin основано на вычислении хеш-функции SHA-256, где 256 и есть длина входа. Если на стадии формирования системы она работала на базе обычных персональных компьютеров, затем на базе мощных видеокарт, то сейчас основную роль играют специально сконструированные «фермы», предназначенные ровно для вычисления SHA-256 и ни для чего другого.

5. Параллелизм. Некоторые задачи «хорошо параллелятся», то есть разбиваются на подзадачи, выполнение которых не зависит друг от друга. Если можно запускать подзадачи на разных машинах, то общее время работы существенно уменьшится. Поэтому изобретение алгоритмов для параллельных вычислений или доказательство, что таковых нет, весьма важно для практики. Именно за счёт более успешного параллелизма растёт производительность современных процессоров.

6. Длина программы. Бывают ситуации, когда программа с более длинным текстом будет работать быстрее. Поэтому ограничения на длину программы могут быть важны с точки зрения расхода остальных ресурсов. Изучению этого фактора посвящена теория колмогоровской сложности, в том числе с ограничением на ресурсы.

7. Интерактивность. В современном мире всё большее распространение получают облачные вычисления и другие подходы с делегированием вычислительных мощностей. За счёт подобной интерактивности, с одной стороны, можно получить доступ к большим вычислительным ресурсам, с другой, приходится доверяться удалённому вычислителю. Оказывается, часто проверка результата может быть значительно проще его получения, поэтому можно использовать интерактивность без опасений. Интересно, что интерактивность гораздо эффективнее работает в сочетании со случайностью. На этом основана теория интерактивных доказательств. Мерой интерактивности можно назвать общее число переданных битов, называемой коммуникационной сложностью протокола.

8. Спецшальние модели вычислении. Существуют модели вычислений, которые пока что не умеют симулировать на стандартных компьютерах без значительного роста вычислительных ресурсов. Прежде всего, это недетерминированные, альтернирующие и квантовые вычисления. Некоторые задачи на данный момент решаются в этих моделях гораздо быстрее, чем в классической. Квантовые вычисления остаются за рамками нашего рассмотрения.

9. Оракул. Вычисления с оракулом позволяют про любое слово за один шаг узнавать, принадлежит ли это слово некоторому фиксированному множеству. Как известно, это даже расширяет класс алгоритмически решаемых задач. Также это расширяет класс задач, решаемых эффективно, а количество запросов к оракулу

можно само по себе воспринимать как меру эффективности алгоритма. Вычисления с оракулом отличаются и от вычислений с подсказкой: при вычислениях на разных словах запросы к оракулу могут быть разными, – и от интерактивных: ответ оракула на повторно заданный вопрос будет таким же, как в первый раз.

Возможно, в будущем будут изучаться и другие важные на практике ресурсы, такие как электроэнергия, рабочее время программиста или общие финансовые затраты.

Эти ресурсы важны не столько для отдельных алгоритмов, сколько для распределённых систем, таких как криптовалюты. Например, Bitcoin отличается крайне низкой средней энергетической эффективностью; по состоянию на сентябрь 2017 года одна транзакция требует около 100 кВт ч. При этом нельзя сказать, что такие затраты объективно нужны для решаемых задач, они возникают из-за особенностей конкретной реализации.

Пока таких теорий не создано.

Комбинационная и описательная сложность относятся к задачам проектирования вычислительных устройств, работающих согласно заданной логике. Одной из актуальных современных задач такого типа является реализация логики работы процессора вычислительного устройства на кристалле минимального размера.

Комбинационная сложность относится не к области программирования алгоритмов, а к области конструирования вычислительных схем [31].

Рассмотрим операции and (логическое «и»), or (логическое «или»), xor (логическое «исключающее или») на двумя битами, результатом выполнения которых является один бит:

x_1x_2	$\mathrm{and}(x_1,x_2)$	x_1x_2	$\mathrm{or}(x_1,x_2)$	x_1x_2	$\mathrm{xor}(x_1,x_2)$
00	0	00	0	00	0
01	0	01	1	01	1
10	0	10	1	10	1
11	1	11	1	11	0

Множество значений одного бита есть $B = \{0,1\}$. Множество значений пары бит обозначают как $B^2 = \{00,01,10,11\}$. Указанные операции and, or, xor имеют вид $f: B^2 \to B$. Тот факт, что функция f получает на вход элемент множества X, а возвращает в качестве результата элемент множества Y, коротко записывают как $f: X \to Y$.

Из этих элементов можно конструировать различные схемы, которые на вход получают m бит, а возвращают n бит, то есть являются реализацией функций вида $B^m \to B^n$. Такие функции называются булевыми функциями.

Например, функцию

$$f(x_1,x_2,x_3) = \mathrm{or}\big(\mathrm{and}(x_1,x_2),\mathrm{and}(x_2,x_3)\big)$$

вида $B^4 \to B^1$ можно визуально представить в виде схемы, показанной на рисунке 53.

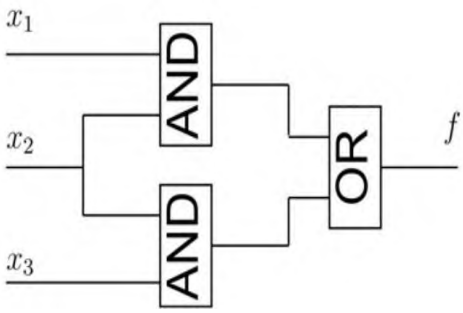


Рисунок 1. Пример реализации логической функции в виде схемы

Возникают следующие вопросы:

1. Любую ли булеву функцию можно реализовать с помощью данного набора элементов?

2. Какое минимальное число элементов требуется для реализации некоторой данной булевой функции $f: B^m \to B^n$?

Ответ на первый вопрос отрицательный. Не все булевы функции можно реализовать в виде композиции элементов and, or, xor. Чтобы иметь возможность описывать все функции, необходимо добавить элемент not : $B \to B$, который 0 отображает в 1 , а 1 отображает в 0 , а также специальный элемент, обозначаемый как 1, который ничего не получает на вход, а возвращает значение 1. Наборы элементов, с помощью которых можно реализовать (в виде схемы) произвольную булеву функцию, называются полными. В частности набор {and, or, not, 1} является полным. Также полным является набор {and, not, 1}.

Второй вопрос непосредственно касается комбинационной сложности.

Комбинационная сложность булевой функции f относителъно набора элементов {and, or, not, 1} – это минималъное число элементов указанного типа, с помощью которых можно реализовать функиию f в виде схемы.

Безусловно, можно определять комбинационную сложность относительно произвольного набора элементов. Обычно берут полные наборы, чтобы сложность была определена для всех элементов.

Понятие комбинационной сложности определяется не только на множестве булевых функций. Его можно определять, например, на множестве натуральнозначных функций натурального аргумента, то есть функций вида $f: \mathbb{N} \to \mathbb{N}$. В качестве элементарных элементов можно взять функции умножения, сложения, вычитания, а также элементы без входов – $\mathbf{0}$ и $\mathbf{1}$, которые всегда равны числам 0 и 1, соответственно.

Таким схемам естественно сопоставить арифметические выражения (рис. 54). Но при этом, число операций в арифметическом выражении, возможно, будет больше, чем число элементов в соответствующей схеме. Это связано с тем, что в схеме выход из каждого элемента можно разветвить на несколько и направить его в

виде входа в несколько других элементов, а в арифметическом выражении подвыражение может «передать» своё значение только в то место, где оно встретилось. Если его значение нужно еще в каком-либо месте, то придётся его просто переписать.

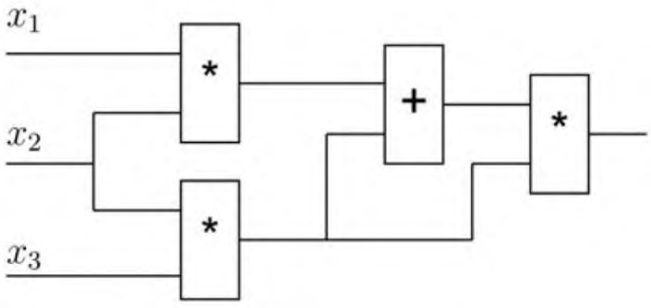


Рисунок 2. Арифметическая схема, вычисляющая выражение $f = \big((x_1 \cdot x_2) + (x_2 \cdot x_3)\big) \cdot (x_2 \cdot x_3)$

Комбинационная сложность относится к задачам проектирования вычислений в железе, в виде процессоров и других электронных вычислительных подблоков. Комбинационная сложность равна минимально возможному размеру схемы, реализующую заданную логику.

Для задачи вычисления арифметических выражений удобно ввести абстрактного исполнителя. Назовём его «Арифмометр». Он получает на вход арифметическое выражение (для него это выражение будет играть роль программы) и последовательно, операция за операцией, согласно расставленным скобкам осуществляет его вычисление. Число операций, которое проделает этот исполнитель (число знаков операций в выражении) соответствует сложности данного выражения-программы.

Задачами для этого исполнителя являются функции, значение которых могут быть записано как арифметическое выражение от аргументов. И сложность задачи для данного исполнителя - это минимальная сложность выражения, которое реализует функцию.

Пусть дан исполнитель и набор элементарных действий, которые он совершает. Результат f может получен в результате последовательности элементарных действий. *Алгоритмическая сложность* f относительно исполнителя – это минимальное число элементарных действий, с помощью которых можно реализовать результат f.

В то время, как комбинационная сложность относится к задачам проектирования устройств (hardware), алгоритмическая сложность относится к задаче программирования алгоритмов.

Здесь важно отметить следующую деталь. В случае, когда исполнитель понимает инструкции подобные циклу while или условному оператору if, количество элементарных действий начинает зависеть от входных данных, и тогда задача поиска

минимального числа элементарных действий в приведённой формулировке теряет смысл.

Например, какова сложность задачи вычисления НОД(a, b) ? Ясно, что время вычисления зависит от входных данных (аргументов a и b). Если зафиксировать входные данные, то пропадает смысл осуществлять какие-либо вычисления - проще просто вернуть заранее вычисленный результат, соответствующий именно этим входным данным.

Для исполнителей, у которых время вычислений может зависеть от входных данных, используется другое определение алгоритмической сложности, а именно, сложностью называется средняя степень роста (асимптотика роста) времени вычислений в зависимости от параметров задачи.

Сложность описания объекта – это длина его описания на некотором формальном языке.

Соответственно, описательная сложность вычислительной задачи относительно заданного формального языка описания алгоритмов – это длина описания самой короткой программы, которая решает данную задачу.

Эта сложность, также как во всех предыдущих случаях, имеет относительный характер она зависит от выбранного исполнителя (языка описания алгоритмов).

Длина программы – не самый важный показатель её эффективности. Безусловно, степень использования ресурсов компьютера (процессорного времени, памяти, сетевых устройств, …) является более важным показателем с практической точки зрения. И, конечно, не следует стремится писать короткие программы. Гораздо важнее, чтобы они были эффективными, понятными, правильно структурно организованными.

Но бывают задачи, где сложность описания играет существенную роль. В частности, графические изображения можно рассматривать как результат выполнения «рисующих» алгоритмов, записанных на некотором специальном языке. Существуют целый ряд интересных алгоритмов сжатия, основанных на том, что программа сжатия старается найти алгоритм как можно меньшего размера, выполнения которого приводит к данному изображению. В этом случае не так важно, сколько времени будет работать алгоритм, а важно, чтобы описание алгоритма было как можно меньше.

1.2. Асимптотический метод оценки вычислительной сложности алгоритмов

Поговорим о том, в каких единицах измерять сложность алгоритма [32]. Мы уже перечислили некоторое количество вычислительных ресурсов. Во многих случаях единицы измерения очевидны из определения: число использованных случайных битов, число параллельно работающих машин, длина подсказки, число запросов к оракулу и т.д. В других случаях выбор бинарный, поэтому никаких единиц нет.

Например, квантовые вычисления, интерактивность или возможность адаптивных запросов к оракулу либо доступны, либо нет. А вот к определению времени работы и памяти возможны разные подходы.

Как правило, в вычислительных моделях есть понятие элементарного шага и элементарной ячейки памяти. Соответственно, именно эти элементарные единицы используются для подсчёта времени работы и памяти. Этот подход называется битовой сложностью. В некоторых моделях возможны более сложные подходы. Например, может подсчитываться количество алгебраических операций над натуральными числами (отдельно аддитивных и мультипликативных, этот подход называется алгебраической сложностью), или необходимое число регистров, в которых также хранятся натуральные числа. Ярким примером алгебраического подхода является задача о быстром умножении матриц. Вычисление произведения двух матриц $n \times n$ по определению требует $O(n^3)$ мультипликативных операций: в каждой из n^2 ячеек нужно перемножить n пар чисел и потом сложить. Алгоритм Штрассена снижает это число до $O\left(n^{\log_2^7}\right) \approx O(n^{2,807})$. Аддитивных операций при этом становится больше, но асимптотически этим можно пренебречь, тк. сложение быстрее умножения. Алгоритм Копперсмита–Винограда снижает число мультипликативных операций до $O(n^{2.375})$, недавние достижения ещё чуть-чуть улучшили эту оценку. Алгебраический подход позволяет выделить самое важное: во-первых, мультипликативные операции производятся значительно дольше аддитивных, и сокращение числа умножений за счёт увеличения числа сложений оправдано. Во-вторых, мы отделяем количество операций с элементами матрицы от сложности осуществления отдельной такой операции, которую на практике можно считать константой, а в теории она даст лишние полилогарифические множители.

Пусть фиксированы конкретный алгоритм и конкретная мера ето сложности, например, число шагов машины Тьюринга. Как из сложности работы на каждом входе получить сложность всего алгоритма? Во-первых, для слов одинаковой длины можно использовать два подхода: подсчитывать сложность в худшем случае, измеряемую как максимум по всем входам данной длины, и сложность в среднем, для которой эти величины так или иначе усредняются. Конкретный выбор зависит от приложений: если мы интересуемся, есть ли универсальный эффективный алгоритм для всех экземпляров задачи, то нужно считать сложность в худшем случае. А если мы хотим использовать сложность задачи, например, для построения стойкого шифра, то нужно считать сложность в среднем. Возможно также изучение сложности в типичном случае: на входах, которые «обычно» встречаются на практике. Но такой подход трудно формализовать. Во-вторых, разных длин тоже бесконечно много, поэтому мы будем изучать асимптотическое поведение сложностной меры при длине, стремящейся к бесконечности. У такого подхода есть недостатки, ведь асимптотическое поведение может быть достигнуто только при нереальных

значениях длины входа. Таких примеров немало, но цельную теорию трудно построить иначе.

Как было сказано, алгоритмы должны обладать свойством массовости, то есть получать входные данные и осуществлять с ними-либо преобразования (вычисления) [31]. Время вычислений в общем случае зависит от входных данных.

Например, в задаче «Ханойские башни» для перемещения n дисков требуется время $2^n - 1$. В этой задаче входом является число n и время вычисления растёт примерно как n-я степень двойки. В таких случаях говорят, что асимптотическая сложность задачи есть $\Theta(2^n)$. Запись $\Theta(f)$ используют тогда, когда речь идёт о функции, примерно пропорционалвной f.

Для того, чтобы освоится с понятием асимптотической сложности, давайте оценим, как растёт время вычисления НОД (a, b) в рекурсивном улучшенном алгоритме Евклида.

```
1: function НОД(a, b)
2:     if a > b and b > 0 then
3:         return НОД(b, min(a mod b, b − a mod b))
4:     else if a < b then
5:         return НОД(b, a)
6:     else
7:         return a
8:     end if
9: end function
```

Выражение $(a \mod b)$ означает остаток при делении a на b. Операция mod имеет больший приоритет, чем операции сложения и вычитания. Выражение $\min(a \mod b, b - a \mod b))$ равно расстоянию на числовой прямой от числа a до ближайшего числа, кратного b.

Обратите внимание на то, что при вычислении НОД (a, b) рекурсивно вызывается вычисление функции НОД с другими значениями аргументов, причём первый аргумент a заведомо больше b. Это будет верно для всех последующих рекурсивных вызовах и условие оператора if будет выполнено. В результате при каждом рекурсивном вызове новое значение a будет равно старому b, а новое значение b - расстоянию от старого a до ближайшего числа, кратного старому b. Это расстояние меньше либо равно половине старого b. Ясно, что значение b с каждым следующим рекурсивным вызовом становится меньше старого как минимум в два раза. Рекурсивные вызовы прекратятся, когда b достигнет значения 0. Поэтому, можно сказать, что число шагов рекурсии ограничено сверху некоторым числом, которое равно тому, сколько раз нужно разделить исходное b (а точнее $\min(a, b)$) на 2 без остатка, чтобы уменьшить его до 0. Это ни что иное, как округлённые до целого

числа логарифм по основанию 2 от числа b. Поэтому асимптотическая сложность приведённого алгоритма есть

$$O\left(\log_2(\min(a,b))\right)$$

Выражение $O(f)$ используется обозначения класса функций, которые растут пропорционально функции f или слабее, а точнее, ограничены сверху функцией вида $A \cdot f + B$, где A и $B -$ некоторые константы.

1.3. Сравнение наилучших, средних и наихудших оценок. O-, o-, ω- и θ-нотации

Итак, мы анализируем только асимптотическое поведение потраченных ресурсов [32]. Специальные обозначения для асимптотик использовались в теории чисел и математическом анализе с конца XIX века. Точный смысл этих обозначений для информатики был введён Дональдом Кнутом в 1976 году.

Пусть функции f и g отображают натуральные числа в натуральные ненулевые числа. Тогда:

$f(n) = O(g(n))$, если $\exists C \ \ \exists N \ \ \forall n > N \ \ f(n) < Cg(n)$;

$f(n) = \Omega(g(n))$, если $\exists c > 0 \ \ \exists N \ \ \forall n > N \ \ f(n) > cg(n)$;

$f(n) = \Theta(g(n))$, если $\exists c > 0 \ \ \exists C \ \ \exists N \ \ \forall n > N \ \ cg(n) < f(n) < Cg(n)$;

$f(n) = o(g(n))$, если $\forall c > 0 \ \ \exists N \ \ \forall n > N \ \ f(n) < cg(n)$;

$f(n) = \omega(g(n))$, если $\forall C \ \ \exists N \ \ \forall n > N \ \ f(n) > Cg(n)$.

Иными словами, o обозначает асимптотически меньшую скорость роста, O асимптотически не большую, Θ – ровно такую же, Ω – не меньшую, ω – строго большую. Одну и ту же функцию в разных ситуациях можно назвать и $O\big(g(n)\big)$, и $\Omega\big(g(n)\big)$, в зависимости от того, верхняя или нижняя оценка важна для контекста.

Помимо базовых, используются такие синтетические обозначения:

$f(n) = \text{poly}\big(g(n)\big)$, если для некоторой константы c выполнено $f(n) = O((g(n))^c)$;

$f(n) = \text{polylog}\big(g(n)\big)$, если для некоторой константы c выполнено $f(n) = O((\log g(n))^c)$;

$f(n) = \tilde{O}\big(g(n)\big)$, если $f(n) = O\left(g(n)\text{polylog}\big(g(n)\big)\right)$.

Одну и ту же задачу можно решать самыми разными алгоритмами. У каждого из этих алгоритмов есть своя сложность. Естественно ожидать, что для каждой задачи есть оптимальный алгоритм и именно его сложность является сложностью задачи. К сожалению, оптимального алгоритма может не быть: небольшие дополнительные улучшения могут приводить ко всё меньшим и меньшим сложностям, а предельная сложность не будет достигаться никаким конкретным алгоритмом. Теорема Блума показывает, что какую меру сложности ни выбрать, можно подобрать задачу, выглядящую именно так.

Поэтому о сложности задачи всегда говорят в терминах верхних или нижних оценок. Как правило, для задачи есть какой-то алгоритм сложности $O(g(n))$, тогда говорят, что задача лежит в классе с такой сложностью. В гораздо более редких случаях удаётся получить нетривиальную нижнюю оценку: доказать, что любой алгоритм, решающий задачу, имеет сложность $\Omega(h(n))$. Например, для умножения матриц есть тривиальная нижняя оценка $\Omega(n^2)$ и оценка Раца в модели арифметических схем с ограниченными коэффициентами $\Omega(n^2 \log n)$.

Теория сложности вычислений занимается оценкой вычислительной сложности различных вычислительных задач в рамках различных исполнителей [31]. И асимптотическая сложность алгоритмических задач является центральным понятием этой теории.

Сложность вычислимой функции естественно определить через сложность самой лучшей программы для какого-то исполнителя. При этом, как мы уже отметили, можно рассматривать различных исполнителей и различные аспекты сложности: размер и сложность кода программы, требуемую для вычислений память или время вычисления. Время вычисления является наиболее важным аспектом сложности, именно на него обращают внимание в первую очередь при разработке алгоритмов. В качестве эталонного исполнителя обычно рассматривают машину Поста, машину Тьюринга, а чаще, некоторое известное реальное (а не абстрактное) вычислительное устройство.

Здесь мы встречается со следующей проблемой: нельзя определить сложность задачи как сложность самой лучшей программы решающей эту задачу, так как самой лучшей программы может просто не быть: одна программа хороша для входных данных одного типа, а другая - для входных данных другого типа.

Обойти эту трудность помогает рассмотрение асимптотик времени вычисления (или требуемой памяти) как функций от размера входа.

Рассмотрим случай, когда входной и выходной алфавиты равны $B = \{0,1\}$, то есть на вход поступает бинарное слово и результат вычислений - тоже бинарное слово.

Размером входа называется длина двоичного слова на входе.

Пусть P – данная вычислимая функция (задача), а I – некоторый исполнитель. Рассмотрим программу $\mathcal{M}$ для этого исполнителя, которая решает поставленную задачу P.

Пусть $t(\mathcal{M}, x)$ обозначает время работы программы $\mathcal{M}$ на входном слове x.

Определим функцию $T(\mathcal{M}, n)$ максимального времени работы программы $\mathcal{M}$ от размера входа n.

$$T(\mathcal{M}, n) = \max_{|x| \leqslant n, x \in B^*} t(\mathcal{M}, x),$$

где B^* – множество слов в алфавите B; $t(\mathcal{M}, x)$ - время работы программы $\mathcal{M}$ на входе $x \in B^*$; $|x|$ – длина слова x.

Обычно не существует программы, который работает быстрее всех других программ на всех возможных входах.

Можно попробовать найти программу $\mathcal{M}_b$, которая для всех возможных n даёт $T(\mathcal{M}_b, n)$ меньше, чем $T(\mathcal{M}, n)$ для любой другой программы $\mathcal{M} \neq \mathcal{M}_b$. То есть $T(\mathcal{M}_b, n)$ равно

$$T_P(n) = \inf_{\mathcal{M} \in \mathcal{S}(P)} T_{\max}(\mathcal{M}_b, n)$$

Здесь $\mathcal{S}(P)$ есть множество программ, решающих задачу P. Возможные значения времени работы алгоритма обычно дискретное множество на положительной полуоси, поэтому инфимум достигается на некоторой программе $\mathcal{M}_b$. Она в указанном смысле является самой лучшей данного n. Для других значений n программа $\mathcal{M}_b$ будет другой. Это неправильный подход к определению сложности. Определённая указанным образом функция $T_P(n)$ слабо отображает реальную сложность задачи, так как при фиксированном ограничении на размер входа можно написать программу, специально оптимизированную для всех значений входных данных меньше n.

Множество различных «сложностей» - это не множество чисел, а множество функций от длины входа n. Причём естественно рассматривать только положительные неубывающие функции.

Приведём определение верхней и нижней оценки сложности задачи. Пусть дана неубывающая функция $f: \mathbb{N} \to \mathbb{R}$. Сложность задачи P для данного исполнителя меньше (больше) f, если существует (не существует) программа $\mathcal{M}$, время работы которой на входных данных размера меньше n ограничено сверху функцией $f(n)$.

Сложностью задачи можно было бы назвать самую маленькую оценку сверху, или самую большую оценку снизу (особенно хорошо, если эти оценки совпадают). Но дело в том, что естественного линейного порядка на неубывающих функциях нет, и нет возможности выбрать самую большую или самую маленькую функцию из данного множества.

Здесь на помощь может прийти рассмотрение классов эквивалентности положительных неубывающих функций.

Две неубывающие функции f, g асимптотически эквивалентны, если существуют константы A_1, A_2, B_1, B_2 такие, что

$$f(n) < A_1 \cdot g(n) + B_1, \quad g(n) < A_2 \cdot f(n) + B_2.$$

Класс эквивалентности функции $f(n)$ обозначается как $\Theta\big(f(n)\big)$.

В теории алгоритмов часто встречаются следующие классы:

$$\Theta(\log n), \quad \Theta(n), \quad \Theta(n \cdot \log n), \quad \Theta(n^\alpha), \quad \Theta(a^n).$$

Класс $\Theta(f(n))$ можно грубо называть классом функций примерно пропорциональных $f(n)$. Кроме $\Theta(g(n))$ используется ещё символ $O(g(n))$, который означает класс всех функций, которые ограничены сверху, начиная с некоторого $n = N$, функцией $A \cdot g(n) + B$, для некоторых положительных констант A и B.

Есть задачи, для которых их вычислительную сложность естественно связывать с некоторым классом $\Theta(f(n))$. А именно:

Сложность задачи A равна $\Theta(f(n))$ если в классе $\Theta(f(n))$ существуют функиии $h_1(n)$ и $h_2(n)$, являющиеся верхней и нижней оценкой сложности задачи A.

Для некоторых задач такого класса может просто не найтись. В таких случаях приходится рассматривать более крупные классы эквивалентности.

На практике приходится пользоваться иными методами оценки времени работы алгоритмов. Современные программы имеют большое количество параметров скрытых даже в самых элементарных операциях. Детали реализации алгоритма и особенности архитектуры используемого компьютера могут сильно повлиять на время работы алгоритма. Даже оценив асимптотику времени работы программы, тяжело рассчитать реальное время её работы, потому что коэффициенты в оценочной функции сложно вычислить теоретически. От программиста скрыты заложенные в процессор алгоритмы оптимизации выполнения кода, работы с памятью и внешними устройствами. Даже если бы он их знал, формула реалистичной оценочной функции была бы очень сложной. Поэтому на практике пользуются следующими методами сравнения алгоритмов:

1. Тестирование на конечном наборе тестовых примеров, приближённых к реальным входным данным.

2. Профилирование – оценка реального времени работы каждой части программы с последующей интроспекцией (рассмотрением всё более мелких частей программы) и выявлением зависимостей от размера входных данных.

Скорость работы не является единственным критерием отбора алгоритмов. В реальной ситуации также важны простота реализации (сложность описания), расширяемость (возможность расширения класса решаемых задач) и безопасность (защита от некорректных данных и сбоев).

1.4. Подпрограммы

Поясним введённые понятия алгоритмической и комбинационной сложности на примере задачи вычисления (конструирования) рациональных чисел [31]. Рассмотрим вычислителей, для которых множество решаемых задач есть рациональные числа. Назовём их $\mathbb{Q}$-исполнителями и $\mathbb{Q}$-схемами.

Это довольно специальный искусственный случай исполнителей и схем, но он позволяет увидеть особенности различных типов сложности.

$\mathbb{Q}$-исполнитель №1

Пусть наш $\mathbb{Q}$-исполнитель №1 имеет одну ячейку памяти x, в которой может хранится одно рациональное число, и пусть он умеет выполнять две элементарные операции: увеличить число на 1 и обратить число:

$$x \leftarrow x + 1, x \leftarrow 1/x.$$

Изначально в ячейке находится число 0. Будем считать, это в эта ячейка памяти элементарна и может принимать конечное число состояний и в ней могут хранится не все, а лишь некоторые рациональные числа. А именно, будем считать, что модуль числителя и знаменателя ограничены некоторым фиксированным числом. Будем также считать, что операция взятия обратного числа нисколько не стоит.

Алгоритмической сложностью рационального числа q относительно $\mathbb{Q}$-исполнителя №1 назовём минимальное число операций прибавления единицы, необходимых для того, чтобы получить число q.

Интересно отметить, что алгоритмическая сложность числа $q = a/b$, где $a, b \in \mathbb{N}$, в для данного исполнителя в точности равна количеству рекурсивных вызовов при вычислении $\text{НОД}(a, b)$.

Если же считать, что операция добавления единицы ничего не стоит, а операция обращения стоит 1, то сложность оказывается равна количеству рекурсивных вызовов при вычислении по формуле

$$\text{НОД}(a,b) = \begin{cases} a, \text{если } a = b \text{ или } b = 0, \\ b, \text{если } a = 0, \\ \text{НОД}(b, a \mod b), \text{если } a = b. \end{cases}$$

$\mathbb{Q}$ -исполнитель №2 вычисляет арифметические выражения, в которых разрешено использовать число 1, операцию сложения, операцию взятия обратного числа и скобки, для указания приоритета операций. Для удобства исключим 0 из множества задач этого исполнителя.

Алгоритмической сложностью $C_2(q)$ рационального числа q относительно такого исполнителя естественно назвать минимальное число операций в выражении, которое равно q. Например, $q = 5/6 = (1+1)^{-1} + (1+1+1)^{-1}$, поэтому $C_2(5/6) \leqslant 6$ (четыре операции сложения и две операции обращения).

Запишем очевидные соотношения для функции C_2 :

$$\begin{aligned} C_2(q_1 + q_2) &\leqslant C_2(q_1) + C_2(q_2) + 1, \\ C_2(1/q) &\leqslant C_2(q) + 1, \\ C_2(1) &= 0. \end{aligned}$$

Рациональные числа можно получать как выражения, сконструированные из единиц, скобок, операций сложения, и функции $i(x) = 1/x$. Например $6/5 = i\big(i(1+1) + i(1+1+1)\big)$. Минимальное число единиц, в выражении, равном рациональному числу q, назовём *эффективной сложностью рациональных чисел* или просто сложностью рациональных чисел. Обозначим эту сложность как $C(q)$.

Соотношения для эффективной сложности C будут отличаться от соотношений для C_2 :

$$\begin{aligned} C(q_1 + q_2) &\leqslant C(q_1) + C(q_2), \\ C(1/q) &= C(q), \\ C(1) &= 1. \end{aligned}$$

Интересно заметить, что сложность C соответствует минимальному значению единичных резисторов, из которых можно сконструировать параллельно-последовательную схему с сопротивлением равным q.

Кроме того, $C(a/b)$, где $a, b \in \mathbb{N}$, равно минимальному количеству квадратов, на которое можно разрезать прямоугольник со сторонами a и b, если разрезать параллельно сторонам от края до края разрезаемого кусочка.

На рис. 55, а, б показан пример разбиения прямоутольника размера 16 и соответствующая этому разбиению электрическая схема из единичных резисторов.

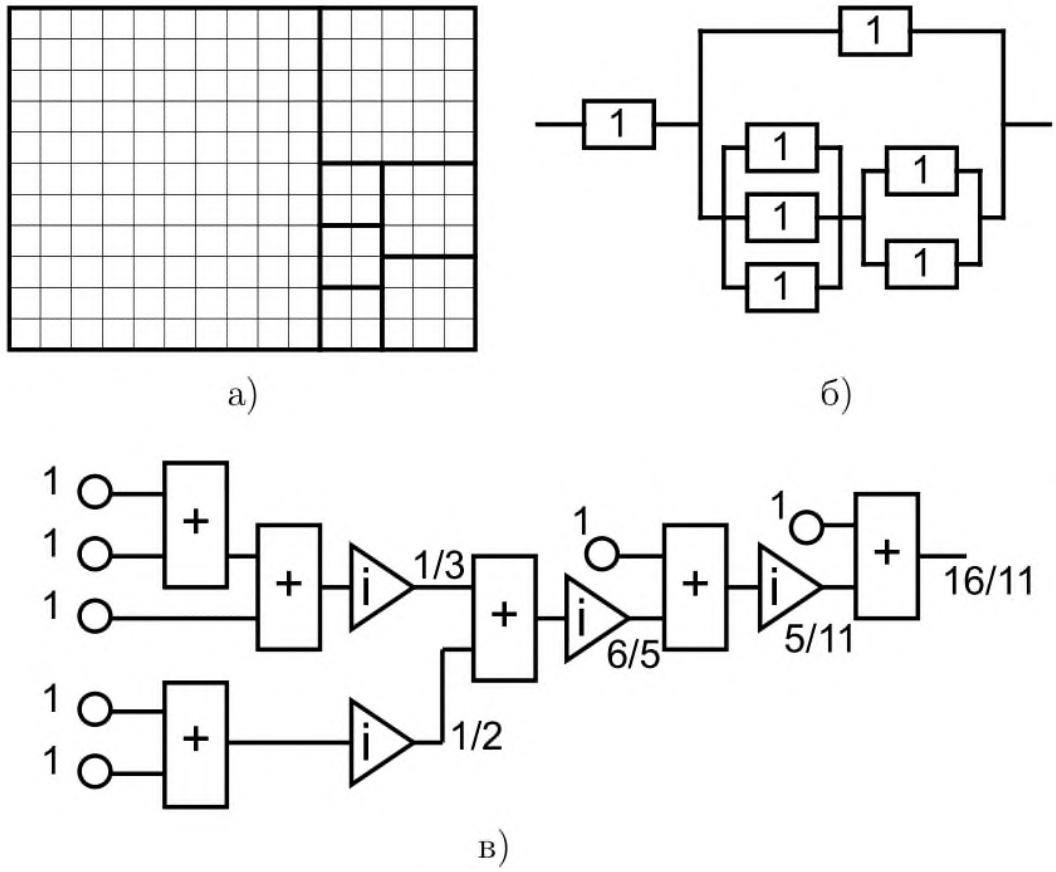


Рисунок 3. Различные способы представления рационального числа $q = 16/11$. *На рисунке (а) показано разбиение прямоугольника с отношением сторон* $16/11$ *на квадраты. На рисунке (б) показана соответствующая разбиению электрическая схема из единичных резисторов с сопротивлением* $R = 1 + (1^{-1} + ((1^{-1} + 1^{-1})^{-1} + (1^{-1} + 1^{-1} + 1^{-1})^{-1})^{-1})^{-1} = 16/11$. *На рисунке (в) показана одна из возможных схем вычисления числа 16/11.*

Рис. 21.3: Различные способы представления рационального числа $q = 16/11$. На рисунке (а) показано разбиение прямоугольника с отношением сторон $16/11$ на квадраты. На рисунке (б) показана соответствующая разбиению электрическая схема из единичных резисторов с сопротивлением $R = 1 + (1^{-1} + ((1^{-1} + 1^{-1})^{-1} + (1^{-1} + 1^{-1} + 1^{-1})^{-1})^{-1})^{-1} = 16/11$. На рисунке (в) показана одна из возможных схем вычисления числа 16/11.

Рассмотрим схемы, в которых провода несут сигнал типа «рациональное число», и можно использовать три элемента: 1 , sum и inverse.

Первый элемент является просто источником единичного сигнала. Обозначим его маленьким кругом. Следующий элемент sum (сумматор) имеет два входа и один выход. Он суммирует два числа, которые получает на вход. Второй элемент inverse, обозначаемый треуголником, имеет один вход и один выход. Он инвертирует рациональное число, то есть меняет числитель и знаменатель местами:

$$\mathrm{sum}(x_1, x_2) = x_1 + x_2, \mathrm{inverse}(x) = 1/x.$$

Например, на рис. 55, в показана схема, вычисляющая $16/11$ по формуле

$$\frac{16}{11} = 1 + \frac{1}{1 + \dfrac{1}{\dfrac{1}{1+1+1} + \dfrac{1}{1+1}}}.$$

В данном случае имеем следующее определение комбинационной сложности рационального числа.

Комбинационной сложностью рационального числа q относительно элементов $\{1$, sum, inverse$\}$ называется минимальное число этих элементов, которых достаточно, чтобы сконструировать схему, вычисляющую q.

Схему вычисления $16/11$ на рис. 55, в можно значительно сократить, если использовать ветвление выходов.

СПИСОК ЛИТЕРАТУРЫ

1. Игошин В.И. Математическая логика и теория алгоритмов : учеб, пособие для студ. высш. учеб, заведений / В. И. Игошин. – 2-е изд. стер. – М.: Издательский центр «Академия». 2008. – 448 с.

2. Глухов М. М., Шишков А. Б. Математическая логика. Дискретные функции. Теория алгоритмов: Учебное пособие. – СПб.: Издательство «Лань», 2012. – 416 с.

3. Геут К. Л. Математическая логика и теория алгоритмов : учсб.-метод. пособие / К. Л. Геут, С. С. Титов. – Екатеринбург : УрГУ ПС, 2017. – 85 с.

4. Мендельсон Э. Введение в математическую логику. – М.: Наука, 1976. – 320 с.

5. Зюзьков В. М. Математическая логика и теория алгоритмов : учебное пособие / В. М. Зюзьков. – Томск : Эль Контент, 2015. – 236 с.

6. Успенский В. Л. Труды но нематематике / В. А. Успенский. 2-е изд,, испр. и доп. : В 5 кн.– М. : ОГИ : Фоад «Магматические этюды», 2012. – Кн. 2 : Философия (с приложением трёх писем П. .П. Трауберг к автору). – 2014. – 566 с.

7. Лихтарников Л. М., Сукачева Т. Г. Математическая логика. Курс лекций. Задачник-практикум и решения: Учебное пособие. 4-е изд., стер. – СПб.: Издательство «Лань», 2009. – 288 с.

8. Вялый М., Подольский В., Рубцов А., Шварц Д., Шень А. Лекции по дискретной математике. 448 с. https://publications.hse.ru/mirror/pubs/share/direct/393719078.pdf

9. Пальчунов, Д. Е. Математическая логика и теория алгоритмов : учеб, пособие / Д. Е. Пальчунов ; Новосиб. гос. ун-т. - Новосибирск, 2016. – Ч. 1. – 96 с.

10. Барашев В.П.. Унучек С.А. Дискретная математика : Учебное пособие Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования «Московский государственный технический университет радиотехники, электроники и автоматики». – М., 2012. –268 с., электронное издание.

11. Чашкин А. В. Дискретная математика : учебник для учреждений высш. проф. образования / А. В. Чашкин. – М.: Издательский центр «Академия», 2012. – 352 с.

12. Гринченков Д.В. Математическая логика и теория алгоритмов для программистов : учебное пособие / Д.В. Гринченков, С.И. Потоцкий. – М.: КНОРУС, 2010. – 208 с.

13. Вайнштейн К. В. Математическая логика и теория алгоритмов : учеб, пособие / Ю. В. Вайнштейн, Т. Г. Пенькова, В. И. Вайнштейн. – Красноярск : Сиб. федер. ун-т, 2019. – 110 с.

14. Новиков Ф. А. Дискретная математика для программистов. Учебник для вузов. 3-е изд. – СПб.: Питер, 2009. – 384 с.

15. Марченков С. С. Основы теории булевых функций. М.: ФИЗМАТЛИТ, 2014. – 136 с.

16. Гаврилов Г. П., Сапоженко А. А. Задачи и упражнения по дискретной математике: Учеб, пособие. – 3-е изд., перераб. – М.: ФИЗМАТЛИТ, 2005. – 416 с.

17. Кормен Т. Х., Лензерсон, Ч. И., Ривест Р. Л., Штайн К. Алгоритмы: построение и анализ. 2-е издание : Пер. с англ. – М. : Издательский дом «Вильямс», 2011. – 1296 с.

18. Алексеев В.В. Теория алгоритмов. Учебно-методическое пособие. СарФТИ НИЯУ МИФИ, 2021. – 100 с.

19. https://education.yandex.ru/handbook/algorithms

20. Ахо А. В., Хопкрофт Д. Э., Ульман Д. Д. Структуры данных и алгоритмы : Пер. с англ. : М. : Издательский дом «Вильямс», 2003. – 384 с.

21. Граничин О.Н. Рандомизированные алгоритмы в задачах обработки данных и принятия решений. Системное программирование. Санкт-Петербургский государственный университет, Т. 6, № 1. С. 141–162.

22. Регеда В. В., Регеда О. Н. Основы алгоритмизации. Методические указания по курсу «Информатика». Пенза: Издательство Пензенского государственного университета, 2008. – 56 с.

23. Туаева М. Т. Алгоритмы. Основы алгоритмизации: Учебное пособие. – Владикавказ: Владикавказский торгово-экономический техникум, 2020. – 51 с.

24. Алгоритмы. Основные алгоритмические конструкции [Текст]: сб. задач / сост. С.А. Рогозин. – Челябинск: Изд-во Челяб. гос. пед. ун-та, 2008. 42 с.

25. Глухова Л.А. Основы алгоритмизации и программирования : лаб. практикум для студ. спец. I-40 01 01 «Программное обеспечение информационных технологий» дневн. формы обуч. В 4 ч. Ч. 3 / Л.А. Глухова, Е П. Фадеева, Е.Е. Фадеева. – Минск : БГУИР, 2007. – 52 с.

26. Макконнелл Дж. Основы современных алгоритмов. 2-е дополненное издание. Москва: Техносфера, 2004. – 368с.

27. Орлов С. А. Теория и практика языков программирования: Учебник для вузов. Стандарт 3-го поколения. – СПб.: Питер. 2013. – 688 с.

28. Анисимов А.Е. Ведение в алгоритмизацию задач / А.Е. Анисимов. – Ижевск: Изд-во «Удмуртский университет», 2017. – 44 с.

29. Конова Е. А. Алгоритмы и программы. Язык С++ : учебное пособие для СПО / Е. А. Конова, Г. А. Поллак. – 3-е изд., стер. – Санкт-Петербург : Лань, 2022. – 384 с.

30. Паклина В.М., Паклина Е.М. Основы алгоритмизации и программирования. – Екатеринбург: ГОУ ВПО УГТУ−УПИ, 2009. – 91 с.

31. Ворожцов А.В., Винокуров Н.А. Лекции Алгоритмы: Построение, анализ и реализация на языке программирования Си. – Москва: МФТИ, 2007. – 452 с.

32. Мусатов Д.В. Сложность вычислений : конспект лекций. – Москва: МФТИ, 2017. – 224 с.

33. Седжвик Р. Фундаментальные алгоритмы на С++. Анализ / Структуры данных / Сортировка / Поиск : Пер. с англ. / Роберт Седжвик. – К.: Издательство «ДиаСофт», 2001. – 688 с.

34. Имран А. 40 алгоритмов, которые должен знать каждый программист на Python. – СПб.: Питер. 2023. – 368 с.

35. Hetland M. L. Python Algorithms: Mastering Basic Algorithms in the Python Language. – 2010. – 337 p.

36. Пинчук В.П. Анализ и построение алгоритмов. Краткий конспект лекций для спец. КСС. – Запорожье: ЗНТУ, 2012.

37. Филиповский В.М. Основы программирования и алгоритмизации. Ч. 2. Практикум по алгоритмизации: Учебное пособие. – СПб.: СПбПУ. 2022. – 71 с.

38. Кормен Т. Х. Алгоритмы: вводный курс. : Пер. с англ. – М. : ООО «И.Д. Вильямс», 2014. – 208 с.

39. Стивенс Р. Алгоритмы. Теория и практическое применение / Род Стивене. – Москва : Издательство «Э». 2016. – 544 с.